

How to Work Like a Professional Engineer (Not Just a Student)

This is the operational layer nobody teaches in a CS degree or a tutorial — the habits that make a hiring manager trust you with real production code. Apply all of this starting with Sprint 1 of Vault.

1. Set up the repo like it's already a team project (even though it's just you)

Before writing any feature code:

- `README.md` from day one — even a rough one. Update it as you go, not at the end. A repo with no README for two weeks is a red flag if anyone looks early.
- `.gitignore` properly configured (`venv`, `.env`, `__pycache__`, etc.) — never commit secrets or environment files
- A `.env.example` file showing what environment variables are needed, without real values
- Decide your folder structure before writing code, not after — restructuring later wastes real time

Why this matters professionally: at a real company, someone else has to be able to clone your repo and run it without asking you questions. If they can't, that's a real cost you're creating for your team.

2. Git discipline — treat every project like it has reviewers, even when it doesn't

Never commit directly to `main`. Practice the real workflow now so it's automatic later:

main (always working, always deployable)

└─ feature/org-scoped-auth

└─ feature/document-upload

└─ fix/celery-worker-crash-on-corrupt-pdf

- One branch per feature or fix, named descriptively
- Open a Pull Request into main even though you're merging your own PRs — write a real PR description: what changed, why, how you tested it
- **Commit message discipline:** `git commit -m "fix"` teaches you nothing and tells a reviewer nothing. Use a real format:

feat: add org-scoped JWT auth dependency

- Adds `require_org_scope` FastAPI dependency
- Enforces `org_id` match on all protected routes
- Returns 403 (not 404) on cross-org access attempts
- Small, frequent commits > one giant commit at the end of a sprint. If you can't describe a commit in one line, it's probably doing too much.

Why this matters professionally: at a real company, your commit history and PR descriptions are how other engineers understand what you did without asking you. Sloppy git history is one of the fastest ways a senior engineer forms a negative first impression of a junior's professionalism, independent of code quality.

3. Break work into tickets, not vibes

Don't sit down and "work on Vault" — that's not a task, it's a mood. Before each sprint:

1. Open a GitHub Issue (or a Projects board) for each concrete task from the sprint plan I gave you
2. Each issue should be small enough to finish in one sitting (a few hours, not days) — if it's bigger, split it
3. Move issues through states: Todo → In Progress → Done — this is literally how real engineering teams track work (Jira, Linear, GitHub Projects all work this way)

Why this matters professionally: the ability to break ambiguous work into scoped, estimable tickets is a skill interviewers explicitly probe for, and it's what separates someone who can work independently from someone who needs constant direction.

4. Write things down — Architecture Decision Records (ADRs)

Whenever you make a real design decision (e.g., "sync vs async ingestion," "org_id as a Qdrant metadata filter vs separate collections per org"), write a short note:

```
## Decision: Org isolation via Qdrant metadata filter, not separate collections
```

Context: need to prevent org A from seeing org B's documents in vector search.

Options considered:

1. Separate Qdrant collection per org
2. Single collection, filter by org_id metadata on every query

Decision: option 2, for now.

Why: fewer moving parts at this scale; revisit if collection size becomes a

performance problem — separate collections would isolate blast radius better

at high scale but add operational overhead we don't need yet.

Put these in a /docs/decisions/ folder, one file per decision.

Why this matters professionally: this is exactly what senior engineers do before big decisions, and exactly what interviewers want to hear when they ask "walk me through a technical decision you made." Having real ADRs from a real project is a strong, concrete answer instead of an improvised one.

5. Code quality tooling — automate the boring judgment calls

Set these up once, at the start of the project:

- **Formatter:** black (Python) — stops arguments about style, just runs

- **Linters:** ruff — catches real bugs (unused imports, undefined names) not just style nits
- **Type hints:** use them on function signatures at minimum — `def get_documents(org_id: int) -> list[Document]:` — this alone catches a surprising number of bugs before runtime, and reads as more senior
- **Pre-commit hook:** run formatter + linter automatically before every commit (pre-commit package, 10-minute setup)

Why this matters professionally: these are table stakes at any company with engineering standards. Not having them is a small thing, but it's a small thing every reviewer notices immediately.

6. How to ask good questions (to me, or to anyone)

Bad: "It doesn't work." Good: "I'm getting a 403 on the `/documents/{id}` endpoint when I expect a 200. Here's the request I'm sending [paste], here's the auth dependency [paste], and here's what I've already checked: the JWT decodes correctly and has the right `org_id`, so I think the bug is in how I'm comparing `org_id` in the dependency, not in the token itself."

The second version is 3x longer to write and saves everyone 10x the back-and-forth. Getting good at this is genuinely a career skill — the engineers who get help fastest are the ones who ask the most precisely.

7. Define "done" correctly

"Done" is not "it worked once on my machine." Done means:

- It has a test (or you've explicitly noted why it doesn't, e.g. "manual-only for now")
- It handles the obvious failure case, not just the happy path
- The README/docs reflect the change if it affects how someone would use the system
- You could explain the decision behind it to someone else in 30 seconds

Getting into this habit now, on a project nobody's grading, is what makes it automatic once someone actually is.

8. Structure your work sessions like sprints, not study sessions

- Before starting a session: pick 1-2 tickets, not "make progress on Vault"
- At the end of a session: write one line in a running log — what you finished, what's blocked, what's next. This is literally what a daily standup is for on a real team, and doing it solo builds the same muscle.
- If you're stuck for more than ~45 minutes on the same error with no new information, that's the signal to ask for help (me, a forum, docs) — not a sign of failure, just an efficient use of time.

Bottom line

None of this is about the code getting smarter — it's about making your *process* look like a professional's, which is exactly what interviewers, reviewers, and future teammates are actually evaluating alongside the code itself. Start every one of these habits on Sprint 1 of Vault, today, not "once the project is more serious" — the habit is the point, and it's much harder to retrofit onto a half-finished project than to build in from commit #1.